

Lab Manual

CSC336-Web Technologies



**Department of Computer Science
Islamabad Campus**

CUI

Lab Contents:

The topics include: HTML; CSS; JavaScript; jQuery/AJAX/JSON; Server-Side Technologies (Node.js/Express); Database Connectivity (MongoDB/Mongoose); REST APIs; React.js; and Web Application Security.

Student Outcomes (SO)

S.#	Description
2	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
3	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations
4	Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations
5	Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings.

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -4	Apply the concepts of markup & scripting languages and client-side technologies.	<i>Applying</i>	2,4
CLO -5	Develop dynamic applications using current industrial practices.	<i>Creating</i>	2-5

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

Table of Contents

Lab #	Main Topic	Page #
Lab 01	HTML Basics, Links and Tables	4
Lab 02	HTML-5 Forms and New Semantic Elements	8
Lab 03	CSS (Cascading Style Sheets) Basics	12
Lab 04	CSS Advanced Concepts	15
Lab 05	Designing Responsive Page Layout using Tailwind CSS / Bootstrap 5	19
Lab 06	JavaScript Basics	23
Lab 07	JavaScript Form Validation, Regular Expressions	27
Lab 08	jQuery, AJAX and JSON	32
	Mid Term Exam	
Lab 9	Request/Response Handling on Server Side (Express.js)	36
Lab 10	Working with Sessions and Middleware	40
Lab 11	Database Connectivity through ORM/ODM (Mongoose/MongoDB)	44
Lab 12	REST API Development and CRUD Operations	48
Lab 13	Creating a React Application and Components	52
Lab 14	Using Hooks, Routing and REST API Integration in React	56
Lab 15	Web Application Security	60
	Final Term Exam	

Lab 01: HTML Basics, Links and Tables

Objectives:

- Understand the basic structure and syntax of an HTML5 document.
- Use heading, paragraph, and text-formatting tags correctly.
- Create ordered and unordered lists.
- Create hyperlinks -internal, external, and email -and distinguish absolute from relative href values.
- Build and format tables, including merged cells using rowspan and colspan.
- Add images and videos to web pages.

Activity Outcomes:

- Design a basic web page using HTML tags.
- Add text formatting tags.
- Add lists to web pages.
- Link pages together using internal, external and email links.
- Build and format tables, including merged header cells.
- Add images and videos to web pages.

Tools / Software Required:

- A text editor -VS Code, Sublime Text, or Notepad++
- A modern web browser -Google Chrome or Mozilla Firefox

Instructor Note: As pre-lab activity, read Chapters 1–3 from "Web Design Playground: HTML & CSS the Interactive Way", Paul McFedries, 2019.

1) Useful Concepts

Tag	Description
<!DOCTYPE>	Defines the document type
<html>	Defines an HTML document
<head> / <title>	Metadata / document title
<body>	Defines the document's body
<h1> to <h6>	Defines HTML headings
<p>, , <hr>	Paragraph; single line break; thematic break
	Creates a hyperlink. Absolute URL (e.g. https://example.com/page.html) points to a full address on the web; relative URL (e.g. about.html or ../images/pic.jpg) is resolved against the current page's location and is the right choice for links within the same site.
, <video src="...">	Inserts an image / video
, ,	Ordered list, unordered list, list item
<dl>, <dt>, <dd>	Description list; term; description

Tag	Description
<table>, <tr>, <th>, <td>	Table; row; header cell; data cell
<thead>, <tbody>, <tfoot>	Groups header / body / footer rows of a table
rowspan, colspan	Merge a cell across multiple rows / columns
<iframe>	Embeds another document within the current page
<!-- ... -->	Comment

HTML Basic Structure:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page Title</title>
  </head>
  <body>
    <!-- page content goes here -->
  </body>
</html>
```

How to create a basic web page:

Step 1: Open a text editor. Step 2: Write your HTML code. Step 3: Save the file with the .html extension, e.g., webpage.html. Step 4: Open the HTML file in a browser. Step 5: Make changes → Save → Refresh the browser.

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	10 Minutes	Low	CLO-4
Activity 2	20 Minutes	Low	CLO-4
Activity 3	30 Minutes	Low	CLO-4
Activity 4	30 Minutes	Low	CLO-4

Activity 1: A personal introduction page

Create a page that introduces yourself. It must include: one <h1> main heading with your name, three <p> paragraphs (about you, your education, and your interests), an unordered list of at least four hobbies, and an ordered list of your top three academic achievements.

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About Muhammad Ali Khan</title>
</head>
<body>
  <h1>Muhammad Ali Khan</h1>
  <p>I am a second-semester BS Computer Science student.</p>
  <p>I completed my intermediate education at ABC College and am now pursuing a degree in Computer Science.</p>
  <p>I am interested in programming, web development, and photography.</p>
```

```

<h2>My Hobbies</h2>
<ul>
  <li>Reading</li>
  <li>Web Development</li>
  <li>Photography</li>
  <li>Cricket</li>
</ul>

<h2>My Top Academic Achievements</h2>
<ol>
  <li>Dean's List, Fall 2025</li>
  <li>1st Position -Intermediate (Pre-Engineering)</li>
  <li>Winner, University Coding Challenge 2025</li>
</ol>
</body>
</html>

```

Activity 2: Adding internal, external and email links

Following Activity 1, add three types of links: internal, external, and email. Also demonstrate the difference between an absolute and a relative link.

Solution:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Muhammad Ali Khan</title>
</head>
<body>
  <h1>Muhammad Ali Khan</h1>
  <h2>My Links</h2>

  <!-- Internal link: an in-page bookmark using a relative fragment reference -->
  <p><a href="#about">Go to About Me</a></p>

  <!-- External link: an absolute URL, points to a different site entirely -->
  <p><a href="https://www.google.com">Visit Google</a></p>

  <!-- Relative link: points to another page in the SAME site/folder, no domain needed -->
  <p><a href="portfolio.html">View My Portfolio</a></p>

  <!-- Email link -->
  <p><a href="mailto:muhammadali@example.com">Email Me</a></p>

  <hr>

  <h2 id="about">About Me</h2>
  <p>My name is Muhammad Ali Khan. I am a second-semester BS Computer Science student.</p>
</body>
</html>

```

Output / Expected behaviour:

```

#about is an internal (relative, same-page) link.
https://www.google.com is an absolute (external) link.
portfolio.html is a relative (same-site) link -it works regardless of domain, which is why site-
internal navigation should normally use relative paths.

```

Activity 3: Adding an educational-history table

Now add a table describing your educational history.

Solution:

```
<table border="1">
  <tr>
    <th>Degree</th>
    <th>Institute</th>
    <th>Year</th>
  </tr>
  <tr>
    <td>BS Computer Science</td>
    <td>ABC University</td>
    <td>2025 - Present</td>
  </tr>
  <tr>
    <td>Intermediate</td>
    <td>ABC College</td>
    <td>2024</td>
  </tr>
  <tr>
    <td>Matriculation</td>
    <td>ABC School</td>
    <td>2022</td>
  </tr>
</table>
```

Activity 4: Adding an image

Following Activities 1–3, add an image to the web page.

Solution:

```

```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Semester Result Card

Create result.html containing a 'Result Card' table with columns Subject, Credit Hours, Marks Obtained, and Grade for five subjects. Use colspan to add a merged title row reading 'Semester Result -Fall 2026' above the column headers.

Lab Task 2: One-page portfolio with bookmarks

Create portfolio.html -a one-page personal portfolio with at least three internal bookmark links (About, Projects, Contact) that jump to sections using id attributes, plus one external link that opens in a new tab (target="_blank").

Lab Task 3: COMSATS library page

Create a webpage for the COMSATS library. Add lists and apply text formatting to your page. Use videos and images of the library and add them to your page.

Lab 02: HTML-5 Forms and New Semantic Elements

Objective:

- Use HTML5 form input types that go beyond plain text.
- Apply built-in HTML5 form validation without writing JavaScript.
- Group related form controls using fieldset and legend.
- Understand and correctly apply HTML5 semantic elements.
- Structure a full webpage layout using semantic tags instead of generic divs.

Activity Outcomes:

- HTML5 input tags
- HTML5 forms
- HTML5 semantic elements

Instructor Note: As pre-lab activity, read Chapter 3 from "Web Design Playground: HTML & CSS the Interactive Way", Paul McFedries, 2019.

1) Useful Concepts

HTML forms pass data to a server. The <form> tag encloses input elements; its key attributes are action (URL to submit to), method ("get" or "post"), and target.

Common input tags:

```
<input type="text" name="textfield" value="initial value">
<textarea name="textarea" cols="24" rows="2">Hello</textarea>
<input type="password" name="password">
<input type="submit" value="Submit"> <input type="reset" value="Reset"> <input type="button"
value="Push Me">
<input type="radio" name="gender" value="male"> Male
<input type="checkbox" name="checkbox" value="checkbox" checked>
<select name="select">
  <option value="red">red</option>
  <option value="green">green</option>
</select>
```

HTML5 attributes and elements:

- required -makes an input field mandatory; pattern -validates input against a regex; readonly / disabled - restrict editing; autocomplete -suggests previous entries.
- <datalist> -provides predefined autocomplete suggestions for a text input.
- type="email", type="date", type="color", type="number" -specialised input fields with built-in validation/UI.

```
<label for="color">My favorite color:</label>
<input type="text" id="color" name="color" list="colors">
<datalist id="colors">
  <option value="Red"><option value="Black"><option value="Blue"><option value="Green">
</datalist>

<input type="email" name="email">
<input type="date" name="dob">
<input type="color" name="favcolor">
<input type="number" name="age" min="18" max="60">
```

HTML5 Semantic elements:

Before HTML5, layout relied almost entirely on generic <div> containers. HTML5 added elements that describe what a section of the page means, improving accessibility and search-engine understanding:

- <header> -introductory content or a site's masthead
- <nav> -a block of primary navigation links
- <main> -the dominant, unique content of the page (one per page)
- <section> -a thematic grouping of content, usually with its own heading
- <article> -a self-contained piece of content, e.g. a blog post
- <aside> -content tangentially related to the surrounding content
- <figure> / <figcaption> -an image (or other media) with an attached caption
- <footer> -footer content for its nearest section or the whole page
- <time datetime="..."> -a machine-readable date or time

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	30 Minutes	Medium	CLO-4
Activity 2	30 Minutes	Medium	CLO-4
Activity 3	30 Minutes	Medium	CLO-4

Activity 1: Identity form

Make a form with name, gender (radio buttons), password and a submit option. Use text boxes for input fields and buttons for the submit option.

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Get Identity</title></head>
<body>
  <p><b>Who are you?</b></p>
  <form method="post" action="">
    <p><label for="name">Name:</label> <input type="text" id="name" name="name"></p>
    <p><label for="password">Password:</label> <input type="password" id="password"
name="password"></p>
    <p>Gender:
      <label><input type="radio" name="gender" value="m"> Male</label>
      <label><input type="radio" name="gender" value="f"> Female</label>
    </p>
    <p><input type="submit" value="Submit"> <input type="reset" value="Reset"></p>
  </form>
</body>
</html>
```

Activity 2: Signup/registration form

Create a student registration form using several HTML5 input types.

Solution:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Student Registration Form</title></head>
<body>
  <h1>Student Registration Form</h1>
  <form method="post" action="">
```

```

    <p><label for="fullname">Full Name:</label> <input type="text" id="fullname" name="fullname"
required></p>
    <p><label for="email">Email:</label> <input type="email" id="email" name="email" required></p>
    <p><label for="password">Password:</label> <input type="password" id="password" name="password"
required></p>
    <p><label for="dob">Date of Birth:</label> <input type="date" id="dob" name="dob"></p>
    <p>Gender:
        <label><input type="radio" name="gender" value="male"> Male</label>
        <label><input type="radio" name="gender" value="female"> Female</label>
    </p>
    <p><label for="color">Favorite Color:</label> <input type="color" id="color" name="color"></p>
    <p><label for="age">Age:</label> <input type="number" id="age" name="age" min="15"
max="60"></p>
    <p>Hobbies:<br>
        <label><input type="checkbox" name="hobby" value="reading"> Reading</label>
        <label><input type="checkbox" name="hobby" value="coding"> Coding</label>
    </p>
    <p><label for="about">About Yourself:</label><br>
        <textarea id="about" name="about" rows="4" cols="40"></textarea></p>
    <p><input type="submit" value="Register"> <input type="reset" value="Reset"></p>
</form>
</body>
</html>

```

Activity 3: Using semantic elements

Recreate the layout of a simple news/blog homepage using only semantic tags: a <header> containing a site title and a <nav> with four links; a <main> containing two <article> elements, each with its own <figure>/<figcaption>; one <aside> listing three 'Related Links'; and one <footer> with a copyright line built using <time>.

Solution:

```

<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Campus Times</title></head>
<body>
    <header>
        <h1>Campus Times</h1>
        <nav>
            <a href="#">Home</a> <a href="#">Sports</a> <a href="#">Tech</a> <a href="#">Contact</a>
        </nav>
    </header>
    <main>
        <article>
            <h2>University Hosts Final Year Project Exhibition</h2>
            <figure>
                
                <figcaption>Students showcase final-year projects.</figcaption>
            </figure>
            <p>The Computer Science department held its annual Final Year Project Exhibition...</p>
        </article>
        <article>
            <h2>Football Team Wins Inter-University Cup</h2>
            <p>The university football team secured first place...</p>
        </article>
    </main>
    <aside>
        <h3>Related Links</h3>
        <ul><li><a href="#">Admissions</a></li><li><a href="#">Events Calendar</a></li><li><a
href="#">Alumni Network</a></li></ul>
    </aside>
    <footer>
        <p>&copy; <time datetime="2026">2026</time> Campus Times. All rights reserved.</p>

```

```
</footer>  
</body>  
</html>
```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: File upload and search field

Extend registration.html by adding a file-upload field for a CV (accept only .pdf and .doc files) and a search field styled with type="search" and a placeholder of 'Search programs...'.

Lab Task 2: Personal profile page using semantic elements

Create a personal profile page using different HTML5 semantic elements, as demonstrated in Activity 3.

Lab 03: CSS (Cascading Style Sheets) Basics

Objective:

To introduce Cascading Style Sheets (CSS) and its role in controlling presentation, layout, typography, colors, spacing, and interactive behaviour of HTML documents.

Activity Outcomes:

- Apply CSS using inline, internal, and external stylesheets.
- Understand CSS selectors and declaration syntax.
- Use common CSS properties for typography, colors, spacing, borders, and backgrounds.
- Understand the CSS cascade, inheritance, and specificity.
- Apply the CSS Box Model to create structured page components.
- Use display, margin, padding, and box-sizing to control layout.
- Style links, buttons, badges, and other interactive elements using :hover and :active.
- Inspect and debug CSS using browser Developer Tools.

1) Useful Concepts

CSS controls presentation -colors, fonts, spacing, borders, backgrounds, positioning -while HTML defines structure. A CSS rule consists of a selector followed by declarations: selector { property: value; }

Three ways to apply CSS:

Method	Where it lives	Trade-off
Inline	style attribute on one element	Quick, but not reusable and hard to maintain
Internal	<style> inside <head>	Fine for a single standalone page; not shareable across pages
External	separate .css file linked via <link>	Reusable, cacheable, cleanest separation of structure/presentation -preferred for production

```
<link rel="stylesheet" href="css/styles.css">

/* styles.css */
* { box-sizing: border-box; }
body { font-family: "Segoe UI", sans-serif; background-color: #f8f9fa; }
.page-title { color: #0d6efd; text-align: center; }
```

Selectors:

```
p { color: #334155; }           /* element selector */
.card { background: white; }    /* class selector: class="card" */
#main-heading { color: blue; }  /* id selector: id="main-heading" */
* { box-sizing: border-box; }   /* universal selector */
```

Cascade and specificity:

When multiple rules target the same element, the browser resolves the conflict using specificity: inline styles > ID selectors > class/attribute/pseudo-class selectors > element/pseudo-element selectors. Among rules of equal specificity, the one appearing later in the stylesheet wins.

```
p { color: black; }           /* element */
.text-sample { color: blue; } /* class */
```

```
#unique-text { color: green; } /* id -wins over the above */
<p id="unique-text" class="text-sample">Text</p> <!-- renders green -->
```

The CSS Box Model:

Every element is a box made of Content → Padding → Border → Margin (innermost to outermost). box-sizing: content-box (default) means width applies only to content, with padding/border added on top; box-sizing: border-box means width includes padding and border, which is far easier to reason about.

```
*, ::before, ::after { box-sizing: border-box; } /* common global reset */

.card { width: 300px; padding: 20px; border: 2px solid #cbd5e1; margin: 30px; }
/* content-box: rendered width = 300 + 20+20 (padding) + 2+2 (border) = 344px */
/* border-box: rendered width = 300px total, padding/border absorbed inside */
```

Display values:

- block -occupies full available width, starts on a new line (div, p, h1, section)
- inline -occupies only the space its content needs (span, a, strong)
- inline-block -stays in line but accepts width/height/margin/padding (buttons, badges)

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	45 Minutes	Medium	CLO-4
Activity 2	45 Minutes	Medium	CLO-4

Activity 3.1: Article-preview component (typography, color, external CSS, links)

Create a professional article-preview component using HTML5 and an external CSS stylesheet, demonstrating typography hierarchy, colors, spacing and interactive link states.

Solution:

```
<!-- index.html -->
<link rel="stylesheet" href="style.css">
<article class="article-card">
  <span class="category-badge">Web Development</span>
  <h1 class="article-title">Mastering Cascading Style Sheets</h1>
  <p class="article-meta">Published on <time datetime="2026-09-09">September 9, 2026</time></p>
  <p class="article-excerpt">CSS gives developers control over webpage presentation...</p>
  <div class="article-footer"><a href="#" class="read-btn">Read Article &rarr;</a></div>
</article>

/* style.css */
*, ::before, ::after { box-sizing: border-box; }
body { margin: 0; padding: 50px 20px; font-family: "Inter", sans-serif; background: #f1f5f9; color: #334155; }
.article-card { max-width: 550px; margin: 0 auto; padding: 32px; background: #fff; border: 1px solid #e2e8f0; border-radius: 8px; }
.category-badge { display: inline-block; padding: 4px 10px; background: #e0e7ff; color: #3730a3; font-size: .75rem; font-weight: 700; text-transform: uppercase; border-radius: 4px; }
.article-title { color: #0f172a; font-size: 1.75rem; font-weight: 700; }
.read-btn { display: inline-block; padding: 10px 20px; background: #0284c7; color: #fff; text-decoration: none; border-radius: 6px; transition: background-color .2s ease; }
.read-btn:hover { background: #0369a1; }
.read-btn:active { background: #075985; }
```

Activity 3.2: Profile card (box model, circular image, hover states)

Create a profile card component demonstrating the CSS Box Model, dimensions, margins, padding, borders, circular images, shadows, and interactive buttons.

Solution:

```
<div class="profile-card">
  <div class="avatar-container"></div>
  <div class="profile-content">
    <h2 class="user-name">Dr. Sarah Khan</h2>
    <p class="user-role">Associate Professor and Researcher</p>
    <button class="action-btn">View Profile</button>
  </div>
</div>

*, *::before, *::after { box-sizing: border-box; }
.profile-card { width: 320px; margin: 0 auto; background: #fff; border: 1px solid #cbd5e1; border-
radius: 12px; box-shadow: 0 10px 25px -5px rgba(0,0,0,.1); overflow: hidden; }
.avatar-img { width: 100px; height: 100px; border: 4px solid #fff; border-radius: 50%; object-fit:
cover; }
.action-btn { width: 100%; padding: 12px; background: #0f172a; color: #fff; border: none; border-
radius: 6px; cursor: pointer; transition: all .2s ease; }
.action-btn:hover { background: #2563eb; transform: translateY(-2px); }
.action-btn:active { transform: translateY(0); }
```

Common CSS errors covered in this lab (for Developer Tools debugging practice):

- Wrong stylesheet path (href="style.css" when the file is actually at css/style.css).
- Unclosed comment (/* ... without a closing */) can swallow following rules.
- Confusing .class and #id selectors for class="..." vs id="..." attributes.
- Forgetting box-sizing: border-box, causing an element's rendered size to exceed its declared width.
- Applying width/height to a naturally inline element (e.g. <a>) without display: inline-block first.

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Personal profile card

Create a professional personal profile card (photo, name, designation, short bio, at least three statistics, contact info, primary action button) using external CSS, class selectors, the box model, box-sizing, border-radius, box-shadow, object-fit, and :hover/:active states.

Lab Task 2: Article-preview component

Create an article-preview component containing category, title, author, publication date, short description and a Read More button, demonstrating typography hierarchy, font weight/size, line-height, and interactive link states.

Lab Task 3: E-commerce product card

Create a product card (image, sale badge, category, name, description, current/original price, stock status, Add to Cart and Wishlist buttons) using external CSS, a global box-sizing: border-box rule, border-radius, box-shadow, overflow: hidden, positioned badge, object-fit, and hover/active/transition states.

Lab 04: CSS Advanced Concepts

Objective:

To develop practical skills in modern CSS layout, responsive design, component positioning, visual effects, and animation, using Flexbox, CSS Grid, transitions, transforms, pseudo-classes/elements, and keyframe animations.

Activity Outcomes:

- Explain when Flexbox and CSS Grid should be used for page and component layout.
- Construct one-dimensional layouts with Flexbox using alignment, spacing, wrapping and ordering.
- Construct two-dimensional layouts with CSS Grid using tracks, gaps, and named areas.
- Apply mobile-first responsive design with media queries and fluid CSS units.
- Use relative, absolute, fixed and sticky positioning appropriately.
- Create interactive visual feedback using pseudo-classes and pseudo-elements.
- Apply transitions, transforms, and controlled @keyframes animations.
- Debug layout and responsive behaviour using browser Developer Tools.

1) Useful Concepts

Flexbox vs Grid:

Feature	Flexbox	Grid
Dimensions	One-dimensional	Two-dimensional
Best for	Component-level alignment (navbars, toolbars, form rows)	Page/section layout (dashboards, galleries, page shells)
Key tools	justify-content, align-items, flex, gap	grid-template-columns, grid-template-areas, gap

```
.navbar { display: flex; align-items: center; justify-content: space-between; gap: 1rem; }  
.card-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 1rem; }
```

Responsive design and fluid units:

A mobile-first approach starts with a compact base layout and progressively enhances it for larger screens using min-width media queries. Prefer content-driven breakpoints over device-named ones.

```
@media (min-width: 768px) { .page { grid-template-columns: 240px 1fr; } }  
@media (min-width: 1100px) { .page { grid-template-columns: 260px 1fr 280px; } }  
  
h1 { font-size: clamp(1.8rem, 4vw, 3.2rem); } /* min, preferred, max */  
.container { width: min(92%, 1100px); padding-inline: max(1rem, 3vw); }
```

Positioning and stacking:

Position	Behaviour	Typical use
static	Normal document flow	Default
relative	Stays in flow; becomes a positioning context	Badge anchors, small offsets
absolute	Removed from flow; positioned against nearest positioned ancestor	Overlays, badges

Position	Behaviour	Typical use
fixed	Positioned relative to the viewport	Persistent action buttons
sticky	Relative until a scroll threshold, then fixed	Sticky headers/sidebars

z-index controls stacking order within a stacking context -a higher value does not automatically place an element above everything else on the page if it sits inside a different stacking context.

Pseudo-classes, pseudo-elements, transitions, transforms:

```
.button:hover { transform: translateY(-2px); }
.button:focus-visible { outline: 3px solid #8ec5ff; outline-offset: 3px; }
input:invalid { border-color: #d73a49; }
.external-link::after { content: " ➔"; font-size: .9em; }

.button { transition: transform 180ms ease, background-color 180ms ease; }
.card:hover { transform: translateY(-6px) scale(1.01); }
```

Keyframe animations and reduced motion:

```
@keyframes pulse { 0%, 100% { transform: scale(1); } 50% { transform: scale(1.05); } }
.status-dot { animation: pulse 1.6s ease-in-out infinite; }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

CSS custom properties (variables):

Custom properties centralise reusable values (colors, spacing, radii) for easier maintenance -a natural next step after the box-sizing/box-model foundation from Lab 03; this lab reuses that foundation rather than re-teaching it.

```
:root { --color-primary: #005cc5; --space-md: 1rem; --radius-md: 0.75rem; }
.card { padding: var(--space-md); border-radius: var(--radius-md); }
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 4.1	30 Minutes	Medium	CLO-4
Activity 4.2	30 Minutes	Medium	CLO-4
Activity 4.3	30 Minutes	Medium	CLO-4

Activity 4.1: Responsive navigation bar

Create a responsive navigation bar. On small screens, navigation links should wrap neatly. On larger screens, the brand, navigation links, and action button should be aligned horizontally.

Solution:

```
<nav class="site-nav">
  <a class="site-brand" href="#">CUI Web Technologies</a>
  <div class="site-links"><a href="#">Home</a><a href="#">Labs</a><a href="#">Contact</a></div>
  <a class="nav-button" href="#">Login</a>
</nav>

.site-nav { display: flex; align-items: center; justify-content: space-between; gap: 1rem; flex-
```



```

wrap: wrap; padding: 1rem 1.5rem; background: #17365d; }
.site-links { display: flex; gap: 1rem; flex-wrap: wrap; }
.nav-button { padding: .55rem .9rem; border-radius: .5rem; background: #005cc5; color: #fff; text-decoration: none; }

@media (max-width: 700px) {
  .site-nav { justify-content: center; }
  .site-brand { flex-basis: 100%; text-align: center; }
}

```

Activity 4.2: Responsive dashboard with CSS Grid

Build a dashboard containing a header, sidebar, main content area, and cards. Use Grid for the page structure and a nested Grid for the cards.

Solution:

```

<div class="dashboard">
  <header class="dashboard-header">Dashboard</header>
  <aside class="dashboard-sidebar">Navigation</aside>
  <main class="dashboard-main">
    <section class="stats">
      <article class="stat-card">Students<br><strong>240</strong></article>
      <article class="stat-card">Projects<br><strong>36</strong></article>
    </section>
  </main>
</div>

.dashboard { min-height: 100vh; display: grid; grid-template: "header header" auto "sidebar main" 1fr / 220px 1fr; }
.dashboard-header { grid-area: header; } .dashboard-sidebar { grid-area: sidebar; } .dashboard-main { grid-area: main; }
.stats { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 1rem; }

@media (max-width: 700px) {
  .dashboard { grid-template: "header" auto "main" 1fr "sidebar" auto / 1fr; }
}

```

Activity 4.3: Interactive card with animation

Create an information card that uses a hover transition, a pseudo-element, and a small keyframe animation. Ensure keyboard users receive visible focus feedback.

Solution:

```

<article class="info-card">
  <h2>CSS Animation</h2>
  <p>Learn how transitions and keyframes improve user interaction.</p>
  <a class="learn-more" href="#">Learn more</a>
</article>

.info-card { position: relative; padding: 1.5rem; border-radius: .9rem; border: 1px solid #d9e2f3; transition: transform 180ms ease; animation: entrance 450ms ease-out both; }
.info-card::before { content: ""; position: absolute; inset: 0 auto 0 0; width: 4px; background: #005cc5; }
.info-card:hover { transform: translateY(-4px); }
.learn-more:focus-visible { outline: 3px solid #8ec5ff; outline-offset: 3px; }

@keyframes entrance { from { opacity: 0; transform: translateY(10px); } to { opacity: 1; transform: translateY(0); } }

```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Responsive university landing page

Create a header, navigation bar, hero section, three feature cards, and a footer. Use Flexbox for the navigation and CSS Grid for the feature section; the layout must adapt to small and large screens.

Lab Task 2: Product card gallery

Create a responsive product gallery with at least six cards (image, name, description, price, badge, action button). Use auto-fit/minmax() for the grid and add an accessible hover/focus interaction.

Lab Task 3: Dashboard layout

Create a responsive dashboard using grid-template-areas (header, sidebar, main, stats cards, footer). On small screens, reorganize into a single-column structure.

Lab Task 4: Animated information component

Create an information component using a pseudo-element, transition, transform, and @keyframes animation. Include a prefers-reduced-motion rule and a visible keyboard-focus state.

Lab Task 5: Refactor a previous lab

Take a page from Lab 01, 02 or 03 and refactor its layout using Flexbox/Grid, introduce CSS custom properties, and make it responsive.

Lab 05: Designing Responsive Page Layouts using Tailwind CSS and Bootstrap 5

Objective:

To enable an interface to adapt its layout, spacing, typography and interactive elements across device sizes, using two contrasting CSS framework approaches: Bootstrap 5 (component-driven) and Tailwind CSS (utility-first).

Activity Outcomes:

- Explain the difference between component-driven and utility-first CSS frameworks.
- Integrate Bootstrap 5 and Tailwind CSS into an HTML document via CDN.
- Construct responsive layouts using Bootstrap's 12-column grid.
- Construct responsive layouts using Tailwind's Grid and Flexbox utilities.
- Apply mobile-first responsive design and breakpoint modifiers in both frameworks.
- Implement hover, focus and active states; build responsive cards, navigation, and product grids.

1) Useful Concepts

Feature	Bootstrap 5	Tailwind CSS
Approach	Component-driven (.btn, .card, .navbar)	Utility-first (flex, p-4, rounded-lg)
Grid	12-column Flexbox grid (.container/.row/.col-*)	CSS Grid / Flexbox utilities (grid-cols-*)
Responsive syntax	col-md-6	md:grid-cols-2
JavaScript	Required for modals/dropdowns/collapses	Generally no framework JS dependency

```
<!-- Bootstrap 5 via CDN -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

<!-- Tailwind Play CDN (for classroom experimentation) -->
<script src="https://cdn.tailwindcss.com"></script>
```

Bootstrap 12-column grid:

```
<div class="container my-5">
  <div class="row g-4">
    <div class="col-12 col-md-6 col-lg-4"><div class="p-4 bg-primary text-white rounded">Column
1</div></div>
    <div class="col-12 col-md-6 col-lg-4"><div class="p-4 bg-secondary text-white rounded">Column
2</div></div>
  </div>
</div>
```

Tailwind Grid and Flexbox:

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  <div class="p-6 bg-indigo-600 text-white rounded-lg shadow">Grid Item 1</div>
  <div class="p-6 bg-violet-600 text-white rounded-lg shadow">Grid Item 2</div>
</div>
```

Responsive breakpoints:

Framework	Sm	md	lg	xl
Bootstrap 5	576px	768px	992px	1200px
Tailwind	640px	768px	1024px	1280px

Important: breakpoint names do not represent identical pixel values across frameworks -always check the framework's own documentation.

Interactive/state modifiers (Tailwind):

```
<button class="bg-slate-700 hover:bg-sky-500 focus:ring-4 focus:ring-sky-500/50 active:scale-95 transition-all">
  Hover & Click Me
</button>
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	30 Minutes	Medium	CLO-4
Activity 2	30 Minutes	Medium	CLO-4
Activity 3	45 Minutes	High	CLO-4

Activity 1: Bootstrap component paradigm

Build a Bootstrap card using `.card`, `.badge`, and `.btn` to demonstrate the component-driven approach.

Solution:

```
<div class="card shadow-sm mx-auto" style="max-width:400px;">
  <div class="card-body">
    <span class="badge bg-primary mb-2">Bootstrap Component</span>
    <h5 class="card-title">Pre-styled UI Element</h5>
    <p class="card-text text-muted">Bootstrap provides semantic component classes like .card and .badge.</p>
    <button class="btn btn-primary w-100">Action Button</button>
  </div>
</div>
```

Activity 2: Tailwind utility-first paradigm

Build the equivalent card using only Tailwind utility classes to contrast with Activity 1.

Solution:

```
<div class="max-w-sm mx-auto bg-white rounded-xl shadow-md border border-slate-200 p-6">
  <span class="inline-block bg-sky-100 text-sky-700 text-xs font-bold px-2.5 py-1 rounded-full mb-3">Tailwind Utility</span>
  <h3 class="text-xl font-bold text-slate-900 mb-2">Atomic Assembly</h3>
  <p class="text-slate-600 text-sm mb-4">Tailwind composes layouts directly in markup using utility classes.</p>
  <button class="w-full bg-sky-600 hover:bg-sky-700 text-white font-semibold py-2 px-4 rounded-lg">Action Button</button>
</div>
```

Activity 3: Responsive e-commerce product catalogue

Develop a responsive product catalogue (navigation header with search/cart, a filter sidebar, a responsive product grid with cards, and a sticky cart summary) using Tailwind CSS.

Solution:

```

<header class="sticky top-0 z-40 bg-slate-900/80 backdrop-blur-md px-4 py-3">
  <div class="max-w-7xl mx-auto flex items-center justify-between">
    <a href="#" class="text-xl font-black text-white">NexusStore</a>
    <input type="text" placeholder="Search products..." class="hidden sm:block bg-slate-800
rounded-xl py-2 px-4 text-sm">
  </div>
</header>

<div class="max-w-7xl mx-auto flex flex-col lg:flex-row gap-8 p-4">
  <aside class="w-full lg:w-64"><!-- category / price filters --></aside>
  <main class="flex-1">
    <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
      <article class="group bg-slate-900 border border-slate-800 rounded-2xl overflow-hidden">
        <div class="relative h-48 overflow-hidden">
          <span class="absolute top-3 left-3 bg-indigo-500 text-white text-[10px] font-black px-2.5
py-1 rounded-full">SALE</span>
          
        </div>
        <div class="p-5">
          <h3 class="font-bold text-white mb-2">Pro Noise-Canceling Headset</h3>
          <div class="flex items-center justify-between">
            <span class="text-lg font-black text-white">$199</span>
            <button class="bg-indigo-600 hover:bg-indigo-500 text-white text-xs font-bold px-3.5
py-2 rounded-xl">Add to Cart</button>
          </div>
        </div>
      </article>
    </div>
  </main>
</div>

<div class="sticky bottom-4 max-w-xl mx-auto w-full px-4">
  <div class="bg-indigo-600/90 text-white p-4 rounded-2xl flex items-center justify-between">
    <div><p class="text-xs">Selected Items (2)</p><p class="font-black">$348.00 USD</p></div>
    <button class="bg-white text-indigo-950 font-black text-xs px-5 py-2.5 rounded-xl">Checkout
Order →</button>
  </div>
</div>

```

Common problems and troubleshooting:

- Unwanted horizontal scrollbar in Bootstrap: a .row is used outside a .container -always wrap rows in a container.
- A Tailwind class like w-[350px] has no effect: check the CDN script is loading, or that a dynamically-built class string isn't being used (Tailwind needs the full class name to appear literally in the markup).
- A Bootstrap navbar toggle does nothing: the bootstrap.bundle.min.js script is missing, or the data-bs-toggle/data-bs-target attributes are missing.

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Responsive multi-column layout comparison

Build the same 3-column responsive layout (stacks on mobile, 2 columns on tablet, 3 on desktop) once using Bootstrap's grid and once using Tailwind's grid utilities; note in comments which took fewer custom CSS lines.

Lab Task 2: Mobile filter drawer

Extend the Activity 3 catalogue so the filter sidebar is hidden on small screens behind a 'Filters' button that toggles its visibility (Bootstrap collapse component or a small Tailwind + vanilla-JS toggle).

Lab Task 3: Responsive navigation menu

Build a responsive navbar (brand, links, and a call-to-action button) that collapses into a mobile menu below a chosen breakpoint, in either framework.

Lab 06: JavaScript Basics

Objective:

To apply core JavaScript (ES6+) concepts and built-in methods when writing reusable, modular programs that process data for web applications.

Activity Outcomes:

- Embed and run scripts; choose declarations and data types; apply operators and control flow.
- Explain scope, hoisting, the temporal dead zone, function forms, this binding and closures.
- Use default/rest parameters, spread, destructuring, objects, arrays, template literals and ES6 modules.
- Select suitable array, string and object methods; build processing chains without unintended mutation.

Required tools: a code editor, an ES6-capable browser with Developer Tools, and a simple local static server (e.g. VS Code Live Server, or node's http-server) so module imports work correctly over HTTP rather than file://.

1) Useful Concepts

Script loading:

```
<script src="activity1.js" defer></script>      <!-- classic script, keeps document order -->
<script type="module" src="app.js"></script>    <!-- ES module, deferred by default -->
```

Variables and data types:

Use `const` when a binding should not be reassigned and `let` when it should change. A `const` object may still have mutable properties -the binding is fixed, not the object's contents.

```
const original = { score: 10 };
const alias = original;
alias.score = 20;           // allowed -mutating the referenced object
console.log(original.score); // 20
// original = {};           // TypeError -reassigning a const binding
```

Operators and coercion:

```
console.log("5" + 2);        // "52": concatenation
console.log("5" - 2);        // 3: numeric coercion
console.log("5" == 5, "5" === 5); // true false
console.log(Number("5") + 2); // 7
```

Scope, hoisting, temporal dead zone:

```
function scopeDemo() {
  console.log(a);           // undefined (var is hoisted)
  var a = 1;
}
// console.log(score);     // ReferenceError: temporal dead zone
let score = 70;
```

Functions, default/rest/spread, destructuring:

```
function add(a, b = 0) { return a + b; }
function sum(...values) { return values.reduce((s, v) => s + v, 0); }
const numbers = [2, 3, 4];
console.log(sum(...numbers));           // 9
const [first, ...rest] = numbers;       // 2 and [3, 4]
const { name, marks = 0 } = { name: "Ayesha" };
```

this, call/apply/bind, closures:

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const a = makeCounter(), b = makeCounter();
console.log(a(), a(), b()); // 1 2 1 -each factory call creates independent state
```

Array processing methods:

Method	Purpose
forEach / for...of	Side effects / iteration
map	New array of transformed values
filter	New array containing matches
reduce	Accumulates to one result
find / some / every	First match / any match / all match
sort	Reorders the array in place (needs a numeric comparator for numbers)

```
const marks = [45, 78, 90, 60];
const labels = marks.filter(m => m >= 50).map(m => `${m}%`);
const ranked = [...marks].sort((a, b) => b - a); // copy first -sort mutates in place
```

ES6 modules:

```
// maths.js
export const passMark = 50;
export function passed(mark) { return mark >= passMark; }

// app.js
import { passMark, passed } from "./maths.js";
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	20 Minutes	Low	CLO-4
Activity 2	25 Minutes	Medium	CLO-4
Activity 3	25 Minutes	Medium	CLO-4
Activity 4	20 Minutes	Medium	CLO-4

Activity 1: Marks summary and control flow

Convert four mark strings to numbers, calculate their total and average, select a result category, and report every mark. Use for...of, while, do...while and switch.

Solution:

```
"use strict";
const rawMarks = ["78", "45", "90", "60"];
const marks = [];
let total = 0;
for (const raw of rawMarks) {
  const value = Number(raw);
  if (raw.trim() === "" || !Number.isFinite(value) || value < 0 || value > 100) {
    throw new Error(`Invalid mark: ${raw}`);
  }
  marks.push(value);
  total += value;
}
```



```

}
const average = total / marks.length;
let result;
if (average >= 80) result = "Excellent";
else if (average >= 50) result = "Pass";
else result = "Needs improvement";
console.log(`Total: ${total}; Average: ${average.toFixed(2)}; Result: ${result}`);
let index = 0;
while (index < marks.length) { console.log(`Student ${index + 1}: ${marks[index]}`); index++; }

```

Output / Expected behaviour:

```
Total: 273; Average: 68.25; Result: Pass
```

Activity 2: Functions, closures and this

Create reusable total/average functions and two independent counters. Demonstrate default/rest parameters, spread, destructuring and call/apply/bind.

Solution:

```

"use strict";
function total(...values) { return values.reduce((s, v) => s + v, 0); }
function createCounter(start = 0) {
  let count = start;
  return { next: () => ++count, current: () => count };
}
const first = createCounter(), second = createCounter(10);
console.log(first.next(), first.next(), second.next()); // 1 2 11

const student = { name: "Ayesha", programme: "BSCS" };
function introduce(prefix) { return `${prefix} ${this.name} (${this.programme})`; }
console.log(introduce.call(student, "Welcome"));
const introduceStudent = introduce.bind(student, "Student:");
console.log(introduceStudent());

```

Output / Expected behaviour:

```

1 2 11
Welcome Ayesha (BSCS)
Student: Ayesha (BSCS)

```

Activity 3: Book catalogue processing

Use a fictional catalogue to compute available books, discounted copies, stock value and search results without altering the original prices.

Solution:

```

"use strict";
const books = [
  { id: 1, title: "HTML Essentials", price: 1000, stock: 5 },
  { id: 2, title: "CSS Layouts", price: 1500, stock: 0 },
  { id: 3, title: "JavaScript Lab", price: 2000, stock: 3 }
];
const available = books.filter(b => b.stock > 0);
const discounted = available.map(b => ({ ...b, price: b.price * 0.9 }));
const value = books.reduce((sum, b) => sum + b.price * b.stock, 0);
const found = books.find(b => b.id === 3);
console.log(`Available: ${available.length}; Stock value: PKR ${value}; Found: ${found.title}`);

```

Output / Expected behaviour:

```
Available: 2; Stock value: PKR 11000; Found: JavaScript Lab
```

Activity 4: A modular marks report

Separate data, processing and presentation across four files: `data.js`, `analytics.js`, `app.js` and `index.html`, loaded with `type="module"`.

Solution:

```
// data.js
export const students = [
  { name: "Ayesha", marks: 78 }, { name: "Bilal", marks: 45 }, { name: "Hina", marks: 90 }
];

// analytics.js
export function summarise(students, passMark = 50) {
  const total = students.reduce((s, x) => s + x.marks, 0);
  return { count: students.length, average: total / students.length,
    passed: students.filter(s => s.marks >= passMark).map(s => s.name) };
}

// app.js
import { students } from "./data.js";
import { summarise } from "./analytics.js";
const { count, average, passed } = summarise(students);
document.getElementById("report").textContent =
  `Students: ${count}\nAverage: ${average.toFixed(2)}\nPassed: ${passed.join(", ")}`;
```

Output / Expected behaviour:

```
Students: 3
Average: 71.00
Passed: Ayesha, Hina
```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Safe marks analyser

Write `analyseMarks(rawMarks)` that converts/validates every input, returns `{ count, total, average, passed, highest }`, uses 50 as the pass threshold, and does not modify the input array. Handle an empty array (all zeros/null) without NaN or division by zero. Throw a clear Error for any blank, non-numeric, non-finite or out-of-range mark.

Lab Task 2: Non-mutating catalogue report

Using the Activity 3 catalogue, implement `buildReport(books, searchText, discount = 0.10)` using `map/filter/reduce/find/some/every/sort` as appropriate, producing normalised titles, case-insensitive search matches, discounted copies, total stock value and an ascending price ranking, without mutating the original array.

Lab Task 3: Reusable revision tracker

Create `tracker.js` exporting `createTracker(start = 0)` with `add(...minutes)`, `current()` and `reset()` methods, keeping state in a closure. Create two independent trackers in `app.js` via a module script and demonstrate that resetting one does not affect the other.

Lab 07: JavaScript Form Validation, Regular Expressions

Objective:

To build interactive web interfaces using DOM manipulation and events, validate and parse form data with regular expressions and JSON, and perform asynchronous data exchange using Promises and the Fetch API.

Activity Outcomes:

- Select, create, update and remove DOM nodes; apply classes/styles and use the event object.
- Explain capturing, bubbling and delegation; distinguish preventDefault from stopPropagation.
- Construct regular expressions using classes, quantifiers, anchors, groups, alternatives, backreferences and flags.
- Validate input with useful error feedback and exchange structured data using JSON.stringify/JSON.parse.
- Trace asynchronous execution and implement Promise chains, Promise.all, async/await and Fetch GET/POST with error handling.

Instructor note: the asynchronous JavaScript activities (4–5) depend on the async/Promises/Fetch lecture; schedule this part of the lab only after that lecture has been delivered, or run a short preparatory demonstration first.

1) Useful Concepts

DOM selection and manipulation:

API	Use
getElementById / querySelector(All)	Select one / the first / all matching elements
createElement, append	Create an element and insert it into a parent
textContent, value	Read/write text safely; read a form control's value
classList.add/remove/toggle	Manage CSS classes without replacing the whole attribute

```
const list = document.querySelector("#topics");
const item = document.createElement("li");
item.textContent = "DOM practice";
list.append(item);
item.classList.add("selected");
```

Events, propagation and delegation:

event.preventDefault() cancels the default action (e.g. form navigation); it does not stop propagation. event.stopPropagation() halts further propagation but does not cancel the default action. Delegation places one listener on a parent and inspects event.target, so it can handle children created later.

```
form.addEventListener("submit", event => {
  event.preventDefault();
  const text = input.value.trim();
  if (!text) { input.focus(); return; }
  // validate / update the DOM / send a request here
});
```

Regular expressions:

Construct	Meaning
[A-Z], \d, \w, \s	Character range; digit; word char; whitespace

Construct	Meaning
*, +, ?, {n,m}	Zero or more; one or more; optional; bounded count
^ and \$	Start and end of input (or line, with the m flag)
(...), (? : ...)	Capturing / non-capturing group
	Alternation
g, i, m flags	Global search; ignore case; multiline anchors

```
const codePattern = /^[1-4][A-Z]{2}\d{2}[A-Z]{2}\d{3}$/;
console.log(codePattern.test("1AB23CD456")); // true

regex.test(text) // boolean
regex.exec(text) // full match + capture groups, or null
text.match(regex) // matches or null
text.replace(regex, "$1") // $1 refers to the first capture group
```

JSON:

```
const json = JSON.stringify({ name: "Ayesha", active: true });
const restored = JSON.parse(json);
// JSON does not preserve methods, undefined values, or circular references
```

Promises, async/await and Fetch:

```
async function getCourses() {
  const response = await fetch("/api/courses");
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}

Promise.all([later("courses", 500), later("settings", 300)])
  .then(values => console.log(values.join(", ")))
  .catch(err => console.log(err.message));
```

fetch() normally resolves even for HTTP errors like 404 -always check response.ok before parsing the body. Parsing (.json()) is itself asynchronous and can fail on malformed data.

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	20 Minutes	Medium	CLO-4
Activity 2	15 Minutes	Medium	CLO-4
Activity 3	30 Minutes	Medium	CLO-4
Activity 4	15 Minutes	Medium	CLO-4
Activity 5	25 Minutes	High	CLO-4

Activity 1: Dynamic topic list with delegated events

Build a list that adds topics, marks them complete and removes them, using one delegated click listener on the parent .

Solution:

```
const form = document.getElementById("topicForm");
const list = document.querySelector("#topics");
```

```

form.addEventListener("submit", event => {
  event.preventDefault();
  const title = document.querySelector("#topic").value.trim();
  if (!title) return;
  const item = document.createElement("li");
  item.innerHTML = `<span>${title}</span>
    <button data-action="done">Done</button>
    <button data-action="remove">Remove</button>`;
  list.append(item);
  form.reset();
});

list.addEventListener("click", event => {
  const button = event.target.closest("button[data-action]");
  if (!button) return;
  const item = button.closest("li");
  if (button.dataset.action === "remove") item.remove();
  else item.classList.toggle("done");
});

```

Activity 2: Regex validation and text processing

Validate a fictional workshop code, extract its components, remove repeated words, split topic tags, and search multiline text.

Solution:

```

const codePattern = /^[1-4][A-Z]{2}\d{2}[A-Z]{2}\d{3}$/;
console.log(codePattern.test("1AB23CD456")); // true

const codeParts = /^([1-4])([A-Z]{2})(\d{2})([A-Z]{2})(\d{3})$/;
console.log(codeParts.exec("1AB23CD456").slice(1).join(" | ")); // 1 | AB | 23 | CD | 456

console.log("web WEB and code code".replace(/\b([A-Za-z]+\s)\b/gi, "$1")); // web and code

const tags = "HTML, CSS ; JavaScript".trim().split(/\s*[;,]\s*/);
console.log(tags); // ["HTML", "CSS", "JavaScript"]

```

Output / Expected behaviour:

```

true
1 | AB | 23 | CD | 456
web and code
['HTML', 'CSS', 'JavaScript']

```

Activity 3: Workshop form validation

Validate name, email, workshop code, and matching passwords with useful error feedback; preview only valid, non-password data as JSON.

Solution:

```

const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const codePattern = /^[1-4][A-Z]{2}\d{2}[A-Z]{2}\d{3}$/;
function passwordOK(v) {
  return v.length >= 10 && /[A-Z]/.test(v) && /[a-z]/.test(v) && /\d/.test(v) && /[!@#%]/.test(v) && !/\s/.test(v);
}
function validate(fields) {
  const errors = {};
  if (fields.fullName.trim().length < 2) errors.fullName = "Enter a name of 2+ characters.";
  if (!emailPattern.test(fields.email.trim())) errors.email = "Enter a valid email.";
  if (!codePattern.test(fields.code.trim())) errors.code = "Use the exact workshop code format.";
  if (!passwordOK(fields.password)) errors.password = "10+ chars, upper/lowercase, digit, symbol,

```

```
no spaces.";
  if (fields.confirm !== fields.password) errors.confirm = "Passwords must match.";
  return errors;
}
// On successful submit: build a payload WITHOUT password/confirm and JSON.stringify it for
preview.
```

Activity 4: Promise sequencing and concurrent work

Demonstrate synchronous vs microtask vs timer-task ordering, a Promise chain with error handling, and Promise.all for concurrent requests.

Solution:

```
console.log("A: synchronous start");
setTimeout(() => console.log("D: timer task"), 0);
Promise.resolve().then(() => console.log("C: promise microtask"));
console.log("B: synchronous end");
// Output order: A, B, C, D

function later(value, delay, fail = false) {
  return new Promise((resolve, reject) => setTimeout(() => fail ? reject(new Error(`Unavailable: ${value}`)) : resolve(value), delay));
}

later(6, 100).then(v => v * 2).then(v => console.log(`Chain result: ${v}`))
  .catch(err => console.log(`Handled: ${err.message}`));

async function loadTogether() {
  const values = await Promise.all([later("courses", 500), later("settings", 300)]);
  console.log(`Together: ${values.join(", ")}`); // input order preserved regardless of which
  resolves first
}
loadTogether();
```

Output / Expected behaviour:

```
A: synchronous start
B: synchronous end
C: promise microtask
D: timer task
Chain result: 12
Together: courses, settings
```

Activity 5: Fetch GET/POST with JSON

Load a list of courses via GET, populate a dropdown, then submit a registration via POST and display the returned record.

Solution:

```
async function requestJSON(url, options = {}) {
  const response = await fetch(url, options);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}

loadButton.addEventListener("click", async () => {
  try {
    const courses = await requestJSON("/api/courses");
    courses.forEach(c => select.append(new Option(c.title, c.id)));
  } catch (err) { status.textContent = `Could not load courses: ${err.message}`; }
});

form.addEventListener("submit", async event => {
```

```

event.preventDefault();
try {
  const saved = await requestJSON("/api/registrations", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: nameInput.value, email: emailInput.value, course: select.value
  })
  });
  status.textContent = `Created demo registration ${saved.id}.`;
} catch (err) { status.textContent = `Submission failed: ${err.message}`; }
});

```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Workshop registration with a review list

Extend Activity 3 with a programme selector (BSCS/BSSE) and a topic-tags field (comma/semicolon separated, trimmed, empty tags discarded). A valid submission creates a review-list entry via safe DOM creation (textContent, never innerHTML with untrusted input), excluding both password fields from the stored/displayed data. Provide one delegated listener for removing entries.

Lab Task 2: Resilient course registration interface

Extend Activity 5 to load courses and a settings.json file concurrently with Promise.all; use the settings' registrationOpen flag to enable/disable the POST button, and handle HTTP, JSON-parsing and network failures distinctly with visible recovery.

Lab 08: jQuery, AJAX and JSON

Objective:

To understand the use of jQuery, AJAX, and JSON in web applications: how jQuery simplifies client-side programming, how AJAX communicates with the server without reloading the page, and how JSON exchanges structured data between client and server.

Activity Outcomes:

- Learn the basic concepts and syntax of jQuery.
- Select and manipulate HTML elements using jQuery.
- Handle events using jQuery.
- Understand AJAX and asynchronous communication between client and server.
- Send and receive data using AJAX.
- Understand the structure and use of JSON data; parse and display it dynamically.

1) Useful Concepts

jQuery is a fast, lightweight JavaScript library that simplifies HTML document traversal, event handling, animation, and AJAX interactions with a concise syntax. AJAX (Asynchronous JavaScript and XML) lets a page send/receive data from a server without a full reload. JSON (JavaScript Object Notation) is the lightweight data format most commonly exchanged over AJAX.

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	20 Minutes	Medium	CLO-4
Activity 2	20 Minutes	Medium	CLO-4
Activity 3	20 Minutes	Medium	CLO-4
Activity 4	20 Minutes	Medium	CLO-4
Activity 5	25 Minutes	Medium	CLO-4
Activity 6	25 Minutes	Medium	CLO-4

Activity 1: jQuery event handling and DOM manipulation

Create buttons to hide an element, show an element, and change the text of an element.

Solution:

```
<h2 id="title">jQuery Event Handling</h2>
<p id="para">jQuery makes JavaScript programming easier.</p>
<button id="hide">Hide</button>
<button id="show">Show</button>
<button id="change">Change Text</button>

<script>
  $(document).ready(function() {
    $("#hide").click(function() { $("#para").hide(); });
    $("#show").click(function() { $("#para").show(); });
    $("#change").click(function() { $("#para").text("Text has been changed using jQuery!"); });
  });
</script>
```



```
});
</script>
```

Activity 2: DOM manipulation and validation using jQuery

Create a form with Name and Email fields. Use jQuery to validate that both fields are not empty when the user submits the form.

Solution:

```
<form id="userForm">
  <label for="name">Name:</label><input type="text" id="name"><br>
  <label for="email">Email:</label><input type="email" id="email"><br>
  <button type="submit">Submit</button>
</form>
<p id="error" style="color:red;"></p>

<script>
$(document).ready(function() {
  $('#userForm').submit(function(event) {
    event.preventDefault();
    const name = $('#name').val(), email = $('#email').val();
    if (!name || !email) { $('#error').text('Both fields are required.')}
    else { $('#error').text(''); alert('Form submitted successfully!')}
  });
});
</script>
```

Activity 3: Displaying a JSON object with jQuery

Create a JSON object and display its information using jQuery.

Solution:

```
<div id="student"></div>
<script>
$(document).ready(function() {
  const student = { name: "Muhammad Tariq", age: 45, marks: 85 };
  $("#student").html(
    "<p>Name: " + student.name + "</p>" +
    "<p>Age: " + student.age + "</p>" +
    "<p>Marks: " + student.marks + "</p>"
  );
});
</script>
```

Activity 4: Fetch data using AJAX

Create a button that, when clicked, fetches a list of users from a public API and displays the data in a table.

Solution:

```
<button id="fetchUsers">Fetch Users</button>
<table border="1" id="userTable"><tr><th>Name</th><th>Email</th></tr></table>

<script>
$(document).ready(function() {
  $('#fetchUsers').click(function() {
    $.ajax({
      url: 'https://jsonplaceholder.typicode.com/users',
      method: 'GET',
      success: function(data) {
        data.forEach(user => {
          $('#userTable').append(`<tr><td>${user.name}</td><td>${user.email}</td></tr>`);
        });
      }
    });
  });
});
```

```

    },
    error: function() { alert('Error fetching users.')}
  });
});
});
</script>

```

Activity 5: Retrieving JSON data from a local server without reloading

Use AJAX to retrieve JSON data from a small Express server.

Solution:

```

// Server-side code
const express = require('express');
const app = express();
app.use(express.static('public'));
app.get('/students', (req, res) => {
  res.json([
    { name: "Baloch", age: 20, marks: 85 },
    { name: "Ahmed", age: 21, marks: 90 }
  ]);
});
app.listen(3000, () => console.log("Server running at http://localhost:3000"));

// Client-side code (public/index.html)
$("#loadStudents").click(function() {
  $.ajax({
    url: "/students", method: "GET",
    success: function(data) {
      let output = "";
      data.forEach(s => output += `<p>Name: ${s.name} | Age: ${s.age} | Marks: ${s.marks}</p>`);
      $("#result").html(output);
    },
    error: function() { $("#result").text("Unable to retrieve student data."); }
  });
});

```

Activity 6: Submitting a registration form via AJAX

Create a registration form and submit the form data to the server using AJAX.

Solution:

```

// Client-side
$("#registerForm").submit(function(event) {
  event.preventDefault();
  const student = { name: $("#name").val(), email: $("#email").val() };
  $.ajax({
    url: "/register", method: "POST",
    contentType: "application/json", data: JSON.stringify(student),
    success: function(response) { $("#message").html("<h3>" + response.message + "</h3>"); },
    error: function() { $("#message").text("Registration failed."); }
  });
});

// Server-side
const express = require('express');
const app = express();
app.use(express.json());
app.use(express.static('public'));
app.post('/register', (req, res) => {
  console.log("Name:", req.body.name, "Email:", req.body.email);
  res.json({ message: "Registration Successful" });
});

```

```
});  
app.listen(3000, () => console.log("Server running at http://localhost:3000"));
```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Product upload with validation

Create a form to upload a product's details (name, price, description). Use AJAX to send the data to a server, validate it using jQuery, and ensure the price is a positive number.

Lab Task 2: Filterable posts list

Fetch and display a list of posts from a server using AJAX. Implement a search bar to filter posts by title using jQuery.

Lab Task 3: Weather lookup app

Build a weather application where the user enters a city name, and an AJAX request fetches current weather data from an API (e.g. OpenWeatherMap). Display the temperature and weather description dynamically.

Lab 09: Request/Response Handling on Server Side

Objective:

To understand how a server-side application receives requests from clients, processes the requested information, and sends appropriate responses back to the client using Express.js.

Activity Outcomes:

- Learn how Express.js handles HTTP requests and responses.
- Understand the request/response cycle between client and server.
- Learn to handle GET and POST requests.
- Understand how to retrieve data using req.params, req.query, and req.body.
- Learn to send HTML, text, and JSON responses using Express.js.
- Get a first, light look at middleware and a small REST-style route set -these are previewed here and covered in full depth in Lab 10 (Middleware) and Lab 12 (REST API Development), so there is no need to master them yet.

1) Useful Concepts

- HTTP Methods: Express supports GET, POST, PUT and DELETE. GET retrieves information; POST submits data.
- Route Parameters & Query Strings: dynamic values via req.params (/user/:id) or req.query (/search?q=express).
- Middleware (previewed here, full depth in Lab 10): functions that process a request before it reaches its route handler.
- Express Router (previewed here, full depth in Lab 12): enables modular routing across separate files.

```
const express = require('express');
const app = express();

app.get('/home', (req, res) => res.send('Welcome to Home Page'));
app.post('/submit', (req, res) => res.send('Data received successfully'));
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	20 Minutes	Low	CLO-4
Activity 2	20 Minutes	Low	CLO-4
Activity 3 (preview)	30 Minutes	Medium	CLO-4
Activity 4 (preview)	30 Minutes	Medium	CLO-4
Activity 5	30 Minutes	Medium	CLO-4

Activity 1: A basic Express application

Create a basic Express application with GET routes for home, about, and contact pages.

Solution:

```
const express = require('express');
const app = express();
```

```
app.get('/', (req, res) => res.send('Welcome to my Home Page'));
app.get('/about', (req, res) => res.send('About Us Page'));
app.get('/contact', (req, res) => res.send('Contact Us Page'));

app.listen(3000, () => console.log('Server running at http://localhost:3000'));
```

Activity 2: Rendering JSON and HTML, page navigation

Serve HTML from one route, JSON from another, and add a catch-all 404 route.

Solution:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Welcome. <a href="/help">Help</a>'));

app.get('/help', (req, res) => {
  res.send('<h1>Help Page</h1><a href="/">Go to Index Page</a>');
});

app.get('/search', (req, res) => {
  console.log('Query received:', req.query);
  res.send([
    { name: 'Muhammad Umar', age: 15, marks: 93 },
    { name: 'Muhammad Tariq', age: 45, marks: 85 }
  ]);
});

app.get('/*', (req, res) => res.send('404 - Page Not Found'));

app.listen(3000);
```

Activity 3 (preview of Lab 10/Lab 12): Params, query strings, and middleware

Create routes handling dynamic parameters, query parameters, request logging middleware, and a POST route for form submissions.

Solution:

```
app.get('/user/:id', (req, res) => res.send(`User ID: ${req.params.id}`));

app.get('/search', (req, res) => {
  const term = req.query.term || 'No term provided';
  res.send(`Search Results for: ${term}`);
});

app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

const express = require('express');
const app = express();
app.use(express.urlencoded({ extended: true }));
app.post('/submit', (req, res) => {
  const { name, email } = req.body;
  res.send(`Name: ${name}, Email: ${email}`);
});
```

Activity 4 (preview of Lab 10/Lab 12): Router-level middleware and a first REST-style route set

Use middleware specific to one router, and expose a minimal in-memory REST API for users.

Solution:

```
const express = require('express');
const router = express.Router();

router.use((req, res, next) => { console.log('Middleware for users router'); next(); });
router.get('/', (req, res) => res.send('User list'));

const app = express();
app.use('/users', router);

// A minimal REST-style API (developed fully in Lab 12)
app.use(express.json());
const users = [];
app.get('/users', (req, res) => res.json(users));
app.post('/users', (req, res) => { users.push(req.body); res.status(201).json(req.body); });

app.listen(3000);
```

Activity 5: Front-end/back-end registration demo using GET/POST

Build a registration form and process it once with GET, then adapt it to POST.

Solution:

```
<!-- Register.html -->
<body>
  <h1>Registration Form</h1>
  <form action="sending" method="get">
    Name: <input type="text" name="name"><br>
    Email: <input type="email" name="email"><br>
    <input type="submit">
  </form>
</body>

// index.js
const express = require('express');
const app = express();

app.get('/register', (req, res) => res.sendFile(__dirname + '/Register.html'));
app.get('/sending', (req, res) => res.send(req.query));

app.listen(3000);
```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: A small blog application

Create a GET / route for the homepage, a GET /post/:id route to display individual blog posts, and a GET /contact route with a feedback form.

Lab Task 2: CRUD REST API for products

Using an array of objects, implement GET /products, GET /products/:id, POST /products, PUT /products/:id and DELETE /products/:id. (This is deliberately the same scope as Lab 12 -use it here as a first attempt, then compare against the fully-structured version in Lab 12.)

Lab Task 3: Search route with query parameters

Create a GET /search route accepting term and category query parameters, filter an array of products accordingly, return matches as JSON, and display a message when none are found.

Lab Task 4: Navigation menu across routes

Create Home, About and Services routes returning HTML content, with navigation links between them.

Lab Task 5: Registration form using GET and POST

Build a registration form, submit it via GET, display the received data, then change it to POST and use Express middleware to process the POST data.

Lab 10: Working with Sessions and Middleware

Aligns with Lecture 20 (Cookies and Sessions) and Lecture 21 (Middleware). Builds directly on the Express fundamentals from Lab 9.

Objective:

To understand how HTTP statelessness is managed using cookies and sessions, and how Express middleware forms a request/response processing pipeline, including application-level, router-level and error-handling middleware.

Activity Outcomes:

- Explain why HTTP is stateless and how cookies/sessions provide state.
- Set and read cookies using the cookie-parser package.
- Create and destroy sessions using express-session; implement login/logout.
- Write custom application-level and router-level middleware.
- Write centralized error-handling middleware.
- Protect routes using authentication-check middleware.

1) Useful Concepts

Cookies vs Sessions:

A cookie is a small piece of data stored in the browser and sent with every request to the same origin. A session stores state on the server and gives the client only a session identifier (usually via a cookie). Sessions are generally preferred for sensitive state such as login status because the actual data never leaves the server.

```
npm install express express-session cookie-parser
```

Setting up sessions:

```
const session = require('express-session');
app.use(session({
  secret: 'lab10-secret-key',    // use an env variable in production
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 1000 * 60 * 15 } // 15 minutes
}));
```

Middleware signature:

Ordinary middleware: `(req, res, next) => { ... next(); }`. Error-handling middleware has four parameters: `(err, req, res, next)` and must be registered after all routes.

```
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next();
}
app.use(logger); // application-level middleware

// error-handling middleware (note: 4 parameters)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong' });
});
```

Router-level middleware:


```
const router = express.Router();
router.use((req, res, next) => {
  console.log('Middleware scoped to this router only');
  next();
});
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	20 minutes	Low	CLO-4
Activity 2	30 minutes	Medium	CLO-4
Activity 3	25 minutes	Medium	CLO-4

Activity 1: Cookie-based visit counter

Use *cookie-parser* to track how many times a visitor has loaded the page, without using a session.

Solution:

```
const express = require('express');
const cookieParser = require('cookie-parser');
const app = express();
app.use(cookieParser());

app.get('/', (req, res) => {
  let visits = parseInt(req.cookies.visits) || 0;
  visits++;
  res.cookie('visits', visits, { maxAge: 900000 });
  res.send(`You have visited this page ${visits} time(s).`);
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Output / Expected behaviour:

```
First load: 'You have visited this page 1 time(s).'
Reload: 'You have visited this page 2 time(s).'
```

Activity 2: Login/logout using express-session

Create a login form. On correct credentials, start a session and redirect to a protected dashboard route. Add a middleware function *requireLogin* that blocks access to */dashboard* when no session exists.

Solution:

```
const express = require('express');
const session = require('express-session');
const app = express();
app.use(express.urlencoded({ extended: true }));
app.use(session({ secret: 'lab10-secret', resave: false, saveUninitialized: false }));

const USER = { username: 'admin', password: '1234' }; // fictional demo credentials

app.get('/login', (req, res) => {
  res.send(`
    <form method='POST' action='/login'>
      <input name='username' placeholder='Username'><br>
      <input name='password' type='password' placeholder='Password'><br>
      <button type='submit'>Login</button>
    </form>`);
});
```

```

app.post('/login', (req, res) => {
  const { username, password } = req.body;
  if (username === USER.username && password === USER.password) {
    req.session.user = username;
    return res.redirect('/dashboard');
  }
  res.send('Invalid credentials. <a href="/login">Try again</a>');
});

function requireLogin(req, res, next) {
  if (!req.session.user) return res.redirect('/login');
  next();
}

app.get('/dashboard', requireLogin, (req, res) => {
  res.send(`Welcome, ${req.session.user}! <a href="/logout">Logout</a>`);
});

app.get('/logout', (req, res) => {
  req.session.destroy(() => res.redirect('/login'));
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

Output / Expected behaviour:

Visiting /dashboard without logging in redirects to /login.
 After correct login, /dashboard shows the welcome message.
 /logout destroys the session and returns to /login.

Activity 3: Custom logging middleware + centralized error handling

Add a request logger (application-level middleware), a router-level middleware for an /admin router, and a centralized error-handling middleware that catches thrown errors from any route.

Solution:

```

const express = require('express');
const app = express();

// Application-level middleware
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// Router-level middleware
const adminRouter = express.Router();
adminRouter.use((req, res, next) => {
  console.log('Admin router middleware executed');
  next();
});
adminRouter.get('/stats', (req, res) => res.json({ users: 42 }));
app.use('/admin', adminRouter);

// Route that intentionally throws
app.get('/crash', (req, res) => {
  throw new Error('Deliberate failure for demonstration');
});

// Centralized error-handling middleware (4 params, registered last)

```

```
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(500).json({ error: err.message });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Output / Expected behaviour:

```
GET /admin/stats logs 'Admin router middleware executed' then returns { users: 42 }.
GET /crash returns 500 with JSON: { "error": "Deliberate failure for demonstration" }.
```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Session-based shopping cart skeleton

Store an array of item names in `req.session.cart`. Add routes `POST /cart/add` (adds an item), `GET /cart` (lists items) and `POST /cart/clear` (empties the cart). Verify the cart persists across requests but resets after `req.session.destroy()`.

Lab Task 2: Role-based middleware

Extend Activity 2 so `req.session.user` also stores a role ('admin' or 'guest'). Write a `requireAdmin` middleware and protect a `GET /admin/reports` route so only role 'admin' can access it; a 'guest' session should receive 403 Forbidden.

Lab Task 3: Simple rate-limiting middleware

Write a custom middleware (without a package) that counts requests per IP address in memory and responds with 429 Too Many Requests if an IP exceeds 5 requests within 10 seconds. Apply it only to a specific route, e.g. `/search`.

Lab 11: Database Connectivity through ORM/ODM

Aligns with Lecture 22 (Database Connectivity with MongoDB) and Lecture 23 (Object-Document Mapping). This lab uses Mongoose, the standard ODM for MongoDB in Express applications.

Objective:

To connect an Express application to a MongoDB database using Mongoose, define schemas and models, and perform CRUD operations through the ODM instead of raw driver calls.

Activity Outcomes:

- Explain the purpose of an ODM and how a Schema maps to a MongoDB collection.
- Connect an Express app to MongoDB (local or Atlas) using Mongoose and environment variables.
- Define a Mongoose Schema with field types, defaults and validation rules.
- Perform create, read, update and delete operations using Model methods.
- Use projection, sorting, pagination (skip/limit) and basic filtering in queries.
- Write and run a simple seed script.

1) Useful Concepts

```
npm install mongoose dotenv
```

.env file (never commit real credentials):

```
MONGO_URI=mongodb://127.0.0.1:27017/lab11db
```

Connecting to MongoDB:

```
require('dotenv').config();
const mongoose = require('mongoose');

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error('Connection error:', err.message));
```

Defining a Schema and Model:

```
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true },
  marks: { type: Number, min: 0, max: 100, default: 0 },
  createdAt: { type: Date, default: Date.now }
});

const Student = mongoose.model('Student', studentSchema);
module.exports = Student;
```

Common Model methods:

```
Student.create({ name: 'Ayesha', email: 'ayesha@test.com', marks: 78 });
Student.find(); // all documents
Student.find({ marks: { $gte: 50 } }); // filter
Student.findById(id);
Student.findByIdAndUpdate(id, { marks: 90 }, { new: true });
Student.findByIdAndDelete(id);
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	20 minutes	Low	CLO-4
Activity 2	30 minutes	Medium	CLO-4
Activity 3	30 minutes	Medium	CLO-4

Activity 1: Connect and insert documents

Connect an Express app to MongoDB using Mongoose, define a Student schema, and insert two sample documents on server start.

Solution:

```
// models/Student.js
const mongoose = require('mongoose');
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  marks: { type: Number, default: 0 }
});
module.exports = mongoose.model('Student', studentSchema);

// index.js
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const Student = require('./models/Student');
const app = express();

mongoose.connect(process.env.MONGO_URI).then(async () => {
  console.log('MongoDB connected');
  const count = await Student.countDocuments();
  if (count === 0) {
    await Student.insertMany([
      { name: 'Ayesha', email: 'ayesha@test.com', marks: 78 },
      { name: 'Bilal', email: 'bilal@test.com', marks: 55 }
    ]);
    console.log('Sample students inserted');
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Output / Expected behaviour:

```
Console: 'MongoDB connected' then 'Sample students inserted' (first run only).
```

Activity 2: Full CRUD via Mongoose

Expose route handlers that perform create, read, update and delete using the Student model.

Solution:

```
app.use(express.json());

app.post('/students', async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
  } catch (err) { res.status(400).json({ error: err.message }); }
});
```

```

app.get('/students', async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

app.get('/students/:id', async (req, res) => {
  const student = await Student.findById(req.params.id);
  if (!student) return res.status(404).json({ error: 'Not found' });
  res.json(student);
});

app.put('/students/:id', async (req, res) => {
  const updated = await Student.findByIdAndUpdate(req.params.id, req.body, { new: true,
runValidators: true });
  if (!updated) return res.status(404).json({ error: 'Not found' });
  res.json(updated);
});

app.delete('/students/:id', async (req, res) => {
  const deleted = await Student.findByIdAndDelete(req.params.id);
  if (!deleted) return res.status(404).json({ error: 'Not found' });
  res.json({ message: 'Deleted successfully' });
});

```

Output / Expected behaviour:

POST /students with a valid body returns 201 and the saved document (with its `_id`).
 GET /students returns the full array.
 DELETE on an unknown id returns 404.

Activity 3: Validation, filtering and pagination

Add schema-level validation, then support filtering by minimum marks and pagination via query parameters:
 /students?minMarks=50&page=1&limit=2.

Solution:

```

// schema validation example (models/Student.js)
marks: {
  type: Number,
  min: [0, 'Marks cannot be negative'],
  max: [100, 'Marks cannot exceed 100']
}

// route: filtering + pagination + sorting
app.get('/students', async (req, res) => {
  const minMarks = Number(req.query.minMarks) || 0;
  const page = Number(req.query.page) || 1;
  const limit = Number(req.query.limit) || 10;

  const filter = { marks: { $gte: minMarks } };
  const students = await Student.find(filter)
    .sort({ marks: -1 })
    .skip((page - 1) * limit)
    .limit(limit);

  const total = await Student.countDocuments(filter);
  res.json({ page, limit, total, results: students });
});

```

Output / Expected behaviour:

```
/students?minMarks=60&page=1&limit=1 returns the single highest-marks student with marks >= 60,  
plus { page, limit, total }.
```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Product catalogue model

Create a Product schema (name, price, category, stock) with validation (price >= 0, stock >= 0). Implement full CRUD routes as in Activity 2.

Lab Task 2: Referencing another collection

Create an Order schema whose product field stores a mongoose.Schema.Types.ObjectId referencing Product. Implement GET /orders/:id that uses .populate('product') to return the order together with the full product document.

Lab Task 3: Seed script

Write a standalone seed.js (run with node seed.js) that connects to the database, deletes all existing Product documents, and inserts at least 8 sample products. It should print a summary count when finished and close the connection.

Lab 12: REST API Development and CRUD Operations

Aligns with Lecture 25 (REST API Development and Authentication). Builds on the Mongoose model from Lab 11 and restructures it into a proper REST API with a router/controller layout, consistent responses and token-based authentication.

Objective:

To design and implement a complete, well-structured REST API: resource-oriented routes, consistent JSON response shape, correct HTTP status codes, a modular router/controller organisation, and JWT-based authentication for protected endpoints.

Activity Outcomes:

- Apply REST principles: resources, nouns in URIs, HTTP verbs for actions, statelessness.
- Organise routes using `express.Router()` with a separate controller file.
- Return a consistent JSON response envelope and correct status codes.
- Add a 404 handler and a centralized error-handling middleware.
- Issue and verify JSON Web Tokens (JWT) for authentication; protect selected routes.

1) Useful Concepts

REST resource/verb mapping:

HTTP Verb	URI	Action
GET	/api/products	Retrieve all products
GET	/api/products/:id	Retrieve one product
POST	/api/products	Create a product
PUT	/api/products/:id	Replace/update a product
DELETE	/api/products/:id	Delete a product

Consistent response envelope:

```
// success
{ "success": true, "data": { ... } }
// failure
{ "success": false, "error": "Product not found" }

npm install jsonwebtoken bcryptjs
```

Issuing and verifying a JWT:

```
const jwt = require('jsonwebtoken');
const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });

function requireAuth(req, res, next) {
  const header = req.headers.authorization; // 'Bearer <token>'
  if (!header) return res.status(401).json({ success: false, error: 'No token provided' });
  try {
    req.user = jwt.verify(header.split(' ')[1], process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ success: false, error: 'Invalid or expired token' });
  }
}
```



```
}  
}
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	30 minutes	Medium	CLO-4
Activity 2	25 minutes	Medium	CLO-4
Activity 3	30 minutes	High	CLO-5

Activity 1: Full CRUD REST API with consistent responses

Build a complete Products REST API using the response envelope and correct status codes (200, 201, 400, 404).

Solution:

```
// controllers/productController.js  
const Product = require('../models/Product');  
  
exports.getAll = async (req, res) => {  
  const products = await Product.find();  
  res.status(200).json({ success: true, data: products });  
};  
  
exports.getOne = async (req, res) => {  
  const product = await Product.findById(req.params.id);  
  if (!product) return res.status(404).json({ success: false, error: 'Product not found' });  
  res.status(200).json({ success: true, data: product });  
};  
  
exports.create = async (req, res) => {  
  try {  
    const product = await Product.create(req.body);  
    res.status(201).json({ success: true, data: product });  
  } catch (err) { res.status(400).json({ success: false, error: err.message }); }  
};  
  
exports.update = async (req, res) => {  
  const product = await Product.findByIdAndUpdate(req.params.id, req.body, { new: true,  
    runValidators: true });  
  if (!product) return res.status(404).json({ success: false, error: 'Product not found' });  
  res.status(200).json({ success: true, data: product });  
};  
  
exports.remove = async (req, res) => {  
  const product = await Product.findByIdAndDelete(req.params.id);  
  if (!product) return res.status(404).json({ success: false, error: 'Product not found' });  
  res.status(200).json({ success: true, data: {} });  
};
```

Output / Expected behaviour:

Same responses as Lab 11 Activity 2, now wrapped consistently as { success, data } / { success, error }.

Activity 2: Router/controller modularisation, 404 and error handling

Move routes into their own router file, mount it under /api, and add a catch-all 404 handler plus centralized error middleware.

Solution:

```
// routes/products.js
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/productController');

router.get('/', ctrl.getAll);
router.get('/:id', ctrl.getOne);
router.post('/', ctrl.create);
router.put('/:id', ctrl.update);
router.delete('/:id', ctrl.remove);
module.exports = router;

// index.js
app.use(express.json());
app.use('/api/products', require('./routes/products'));

// 404 handler (after all routes)
app.use((req, res) => res.status(404).json({ success: false, error: 'Route not found' }));

// centralized error handler (must have 4 parameters)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ success: false, error: 'Internal server error' });
});
```

Output / Expected behaviour:

```
GET /api/nonexistent returns 404 with { success: false, error: 'Route not found' }.
```

Activity 3: JWT authentication for a protected route

Add a Users collection with hashed passwords, a POST /api/auth/login route that issues a JWT, and protect DELETE /api/products/:id so only an authenticated request may delete a product.

Solution:

```
// models/User.js
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true } // stored as a bcrypt hash
});
module.exports = mongoose.model('User', userSchema);

// routes/auth.js
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const router = express.Router();

router.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username });
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ success: false, error: 'Invalid credentials' });
  }
  const token = jwt.sign({ id: user._id, username }, process.env.JWT_SECRET, { expiresIn: '1h' });
  res.json({ success: true, data: { token } });
});
module.exports = router;
```

```
// applying requireAuth (from Useful Concepts) to one route
const requireAuth = require('../middleware/requireAuth');
router.delete('/:id', requireAuth, ctrl.remove);
```

Output / Expected behaviour:

```
DELETE /api/products/:id without an Authorization header returns 401.
Logging in via /api/auth/login with correct credentials returns a token; sending it as
'Authorization: Bearer <token>' allows the delete to succeed.
```

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Search, filter and pagination on the API

Extend GET /api/products to accept ?search=, ?category=, ?page= and ?limit= query parameters and return matching, paginated results inside the same { success, data } envelope, along with a meta object containing total count and page info.

Lab Task 2: Role-protected delete

Add a role field to the User model ('admin' or 'staff'). Update requireAuth (or add a requireAdmin variant) so DELETE requests are only permitted for role 'admin'; 'staff' tokens should receive 403 Forbidden.

Lab Task 3: API versioning

Restructure the mount points so the existing routes are served under /api/v1/products and /api/v1/auth. Explain, in a short comment block at the top of index.js, why versioning the URI helps when the API's response shape changes later.

Lab 13: Creating a React Application and Components

Aligns with Lecture 26 (Introduction to React.js and Rendering Approaches) and Lecture 27 (Components, Props and Event Handling).

Objective:

To scaffold a React application, understand JSX syntax and rules, and build reusable function components using props, children and basic event handling.

Activity Outcomes:

- Scaffold a React project using Vite and explain the generated folder structure.
- Write JSX and explain how it differs from HTML (className, self-closing tags, one root element).
- Create function components and compose them (parent/child).
- Pass and use props, including default props and children.
- Render lists using map() with a stable key.
- Handle basic DOM events (onClick, onChange) and manage a controlled input.

1) Useful Concepts

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Generated structure (relevant parts):

```
my-app/
src/
  main.jsx      // mounts <App /> into index.html's #root
  App.jsx       // top-level component
  index.html
```

JSX rules to remember:

- A component must return a single root element (or a Fragment <>...</>).
- Use className instead of class; use htmlFor instead of for.
- Curly braces {} embed a JavaScript expression inside markup.
- Every element in a rendered list needs a unique, stable key prop.

A minimal function component:

```
function Greeting({ name = 'Student' }) {
  return <h2>Hello, {name}!</h2>;
}
export default Greeting;
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	20 minutes	Low	CLO-4
Activity 2	30 minutes	Medium	CLO-4
Activity 3	30 minutes	Medium	CLO-4

Activity 1: Scaffold and compose Header/Footer

Create a Vite React project. Build separate Header and Footer components and compose them inside App.

Solution:

```
// src/components/Header.jsx
function Header() {
  return (
    <header>
      <h1>CUI Web Technologies</h1>
    </header>
  );
}
export default Header;

// src/components/Footer.jsx
function Footer() {
  return <footer>&copy; 2026 Department of Computer Science</footer>;
}
export default Footer;

// src/App.jsx
import Header from './components/Header';
import Footer from './components/Footer';

function App() {
  return (
    <>
      <Header />
      <p>Main page content goes here.</p>
      <Footer />
    </>
  );
}
export default App;
```

Output / Expected behaviour:

Page renders the heading, a paragraph and the footer copyright line.

Activity 2: A reusable ProductCard rendered from a list

Create a ProductCard component that accepts name, price and inStock as props. Render an array of products using map(), each with a unique key.

Solution:

```
// src/components/ProductCard.jsx
function ProductCard({ name, price, inStock }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>PKR {price}</p>
      <p>{inStock ? 'In Stock' : 'Out of Stock'}</p>
    </div>
  );
}
export default ProductCard;

// src/App.jsx
import ProductCard from './components/ProductCard';

const products = [
```

```

    { id: 1, name: 'Wireless Headphones', price: 4999, inStock: true },
    { id: 2, name: 'Mechanical Keyboard', price: 8999, inStock: false },
    { id: 3, name: 'USB-C Hub', price: 2499, inStock: true }
  ];

function App() {
  return (
    <div>
      {products.map(item => (
        <ProductCard key={item.id} name={item.name} price={item.price} inStock={item.inStock} />
      ))}
    </div>
  );
}
export default App;

```

Output / Expected behaviour:

Three cards render, each showing its own name, price and stock status.

Activity 3: Controlled form to add a product

Add a simple form with a text input and a submit button; typing updates state (controlled input), and submitting adds a new product to the rendered list without reloading the page.

Solution:

```

import { useState } from 'react';

function App() {
  const [products, setProducts] = useState(['Headphones', 'Keyboard']);
  const [name, setName] = useState('');

  function handleSubmit(event) {
    event.preventDefault();
    const trimmed = name.trim();
    if (!trimmed) return;
    setProducts(prev => [...prev, trimmed]);
    setName('');
  }

  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input value={name} onChange={e => setName(e.target.value)} placeholder="Product name" />
        <button type="submit">Add Product</button>
      </form>
      <ul>
        {products.map((p, i) => <li key={i}>{p}</li>)}
      </ul>
    </div>
  );
}
export default App;

```

Output / Expected behaviour:

Typing 'Mouse' and submitting adds a new 'Mouse' item to the list; the input clears; the page never reloads.

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Student list mini-app

Build a small app with components `StudentList`, `StudentItem` and `SearchBar`. Store an array of student objects in state; `SearchBar` is a controlled input that filters the visible list by name as the user types (case-insensitive substring match).

Lab Task 2: Add and remove items

Extend the student list app so each `StudentItem` has a Remove button that deletes that student from state, and a form (as in Activity 3) that adds new students.

Lab Task 3: Props with children and defaults

Create a `Card` component that accepts a title prop and renders whatever is passed as children inside it. Give title a default value of 'Untitled' and demonstrate the component both with and without a title supplied.

Lab 14: Using Hooks, Routing and REST API Integration in React

Aligns with Lecture 28 (Hooks and State Management) and Lecture 29 (Routing and API Integration). Consumes the REST API built in Lab 12.

Objective:

To fetch data from a REST API using `useEffect/useState` with loading and error states, and to add client-side routing with React Router so the app has multiple pages that share layout.

Activity Outcomes:

- Fetch data inside `useEffect` and handle loading/error/success states correctly.
- Set up React Router with `BrowserRouter`, `Routes`, `Route` and `Link`.
- Read a route parameter with `useParams` to fetch a single resource.
- Navigate programmatically with `useNavigate` after a form submission.
- Send POST requests from a React form to an Express REST API.

1) Useful Concepts

```
npm install react-router-dom axios
```

The data-fetching pattern:

```
const [data, setData] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  let cancelled = false;
  async function load() {
    try {
      const res = await fetch('http://localhost:3000/api/products');
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const json = await res.json();
      if (!cancelled) setData(json.data);
    } catch (err) {
      if (!cancelled) setError(err.message);
    } finally {
      if (!cancelled) setLoading(false);
    }
  }
  load();
  return () => { cancelled = true; }; // cleanup on unmount
}, []);
```

Basic router setup:

```
// src/main.jsx
import { BrowserRouter } from 'react-router-dom';
createRoot(document.getElementById('root')).render(
  <BrowserRouter><App /></BrowserRouter>
);

// src/App.jsx
import { Routes, Route, Link } from 'react-router-dom';
function App() {
  return (
    <>
```



```

    <nav><Link to='/'>Home</Link> | <Link to='/products'>Products</Link></nav>
    <Routes>
      <Route path='/' element={<Home />} />
      <Route path='/products' element={<ProductList />} />
      <Route path='/products/:id' element={<ProductDetail />} />
      <Route path='*' element={<NotFound />} />
    </Routes>
  </>
);
}

```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	25 minutes	Medium	CLO-4
Activity 2	30 minutes	Medium	CLO-4
Activity 3	30 minutes	High	CLO-4

Activity 1: Fetch and display products with loading/error states

Build a *ProductList* component that fetches from the Lab 12 API and clearly shows loading, error and success states.

Solution:

```

import { useEffect, useState } from 'react';

function ProductList() {
  const [products, setProducts] = useState([]);
  const [status, setStatus] = useState('loading'); // loading | error | ready

  useEffect(() => {
    fetch('http://localhost:3000/api/products')
      .then(res => res.json())
      .then(json => { setProducts(json.data); setStatus('ready'); })
      .catch(() => setStatus('error'));
  }, []);

  if (status === 'loading') return <p>Loading products...</p>;
  if (status === 'error') return <p>Could not load products.</p>;

  return (
    <ul>
      {products.map(prod => <li key={prod._id}>{prod.name} -PKR {prod.price}</li>)}
    </ul>
  );
}
export default ProductList;

```

Output / Expected behaviour:

Page briefly shows 'Loading products...', then the list; stopping the Express server and reloading shows 'Could not load products.'

Activity 2: Route parameters -product detail page

Add a route */products/:id* that fetches and displays a single product using *useParams*.

Solution:

```

import { useParams, Link } from 'react-router-dom';
import { useEffect, useState } from 'react';

function ProductDetail() {
  const { id } = useParams();
  const [product, setProduct] = useState(null);
  const [status, setStatus] = useState('loading');

  useEffect(() => {
    fetch(`http://localhost:3000/api/products/${id}`)
      .then(res => {
        if (!res.ok) throw new Error('Not found');
        return res.json();
      })
      .then(json => { setProduct(json.data); setStatus('ready'); })
      .catch(() => setStatus('error'));
  }, [id]);

  if (status === 'loading') return <p>Loading...</p>;
  if (status === 'error') return <p>Product not found. <Link to='/products'>Back</Link></p>;

  return (
    <div>
      <h2>{product.name}</h2>
      <p>PKR {product.price}</p>
      <Link to='/products'>Back to list</Link>
    </div>
  );
}
export default ProductDetail;

// In ProductList, link each item to its detail page:
<Link to={` /products/${prod._id}`}>{prod.name}</Link>

```

Output / Expected behaviour:

Clicking a product name navigates to /products/<id> and shows that product's detail; an invalid id shows the not-found message.

Activity 3: Create-product form with navigation

Add a form that POSTs a new product to the API, then uses useNavigate to return to the product list on success.

Solution:

```

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

function AddProduct() {
  const [name, setName] = useState('');
  const [price, setPrice] = useState('');
  const [error, setError] = useState('');
  const navigate = useNavigate();

  async function handleSubmit(event) {
    event.preventDefault();
    setError('');
    try {
      const res = await fetch('http://localhost:3000/api/products', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name, price: Number(price) })
      });
    }
  }
}

```

```

    const json = await res.json();
    if (!json.success) throw new Error(json.error);
    navigate('/products');
  } catch (err) {
    setError(err.message);
  }
}

return (
  <form onSubmit={handleSubmit}>
    <input value={name} onChange={e => setName(e.target.value)} placeholder='Name' required />
    <input value={price} onChange={e => setPrice(e.target.value)} placeholder='Price'
type='number' required />
    <button type='submit'>Add Product</button>
    {error && <p style={{ color: 'red' }}>{error}</p>}
  </form>
);
}
export default AddProduct;

```

Output / Expected behaviour:

Submitting valid data creates the product on the server and redirects to /products, where the new item now appears.

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Edit and delete from the UI

On ProductDetail, add Edit and Delete buttons. Edit should load the product into a controlled form and PUT the changes; Delete should call the DELETE endpoint and navigate back to /products on success.

Lab Task 2: Not-found route

Add a catch-all <Route path='*' element={<NotFound />} /> that renders a friendly 404 page with a link back to Home for any unmatched URL.

Lab Task 3: Basic loading/error UX polish

Replace the plain 'Loading...' text across all three pages with a small reusable <Status /> component that accepts a state prop ('loading' | 'error' | message) and renders consistent styling; reuse it in ProductList, ProductDetail and AddProduct.

Lab 15: Web Application Security

Aligns with Lecture 30 (Application Security Basics) and Lecture 31 (Common Web Attacks and Defences). Applies directly to the Express/MongoDB API from Labs 11–12 and the React front end from Labs 13–14.

Objective:

To apply core, practical web-application security measures: secure password storage, protection against NoSQL injection and XSS, sensible CORS configuration, common security response headers, and basic rate limiting.

Activity Outcomes:

- Explain the CIA triad and why plaintext password storage is unacceptable.
- Hash and verify passwords using bcrypt.
- Add security headers using helmet and understand what each mitigates.
- Configure CORS explicitly rather than allowing all origins.
- Recognise and fix a NoSQL injection vulnerability in a query.
- Recognise and fix a reflected XSS vulnerability in server-rendered HTML.
- Apply basic rate limiting to a sensitive route.

1) Useful Concepts

Threat	Typical cause	Primary defence used in this lab
Plaintext password exposure	Storing req.body.password directly	bcrypt.hash() before saving; bcrypt.compare() on login
NoSQL injection	Passing untrusted req.body/req.query straight into a Mongo query	Validate/whitelist input types; strip \$ and . operators (e.g. express-mongo-sanitize)
Cross-Site Scripting (XSS)	Inserting untrusted input into HTML without escaping	Escape output server-side; never use dangerouslySetInnerHTML with untrusted data in React
Cross-Origin misuse	cors() with no options (allow-all)	Explicit origin allow-list in the CORS config
Brute-force / abuse	No limit on repeated requests to a route	express-rate-limit on sensitive routes such as /login

```
npm install bcryptjs helmet cors express-rate-limit express-mongo-sanitize
```

Security middleware baseline:

```
const helmet = require('helmet');
const cors = require('cors');
const mongoSanitize = require('express-mongo-sanitize');

app.use(helmet());
app.use(cors({ origin: ['http://localhost:5173'], methods: ['GET', 'POST', 'PUT', 'DELETE'] }));
app.use(mongoSanitize()); // strips keys starting with $ or containing '.'
```

2) Solved Lab Activities

Sr. No.	Allocated Time	Level of Complexity	CLO Mapping
Activity 1	25 minutes	Medium	CLO-4
Activity 2	20 minutes	Medium	CLO-4
Activity 3	30 minutes	High	CLO-5

Activity 1: Hash passwords with bcrypt

Update the registration route from Lab 12 to hash the password before saving, and confirm login still works by comparing hashes.

Solution:

```
const bcrypt = require('bcryptjs');

app.post('/api/auth/register', async (req, res) => {
  const { username, password } = req.body;
  if (!username || !password || password.length < 8) {
    return res.status(400).json({ success: false, error: 'Username and an 8+ character password are required' });
  }
  const hashed = await bcrypt.hash(password, 10);
  const user = await User.create({ username, password: hashed });
  res.status(201).json({ success: true, data: { id: user._id, username: user.username } });
});

// login (unchanged in shape from Lab 12, using bcrypt.compare)
app.post('/api/auth/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username });
  const valid = user && await bcrypt.compare(password, user.password);
  if (!valid) return res.status(401).json({ success: false, error: 'Invalid credentials' });
  res.json({ success: true, data: { username } });
});
```

Output / Expected behaviour:

Inspecting the users collection shows only bcrypt hashes (e.g. '\$2a\$10\$...'), never the plaintext password.

Activity 2: Headers, CORS and rate limiting

Add helmet, an explicit CORS allow-list, and rate limiting on the login route specifically.

Solution:

```
const helmet = require('helmet');
const cors = require('cors');
const rateLimit = require('express-rate-limit');

app.use(helmet());
app.use(cors({ origin: 'http://localhost:5173' }));

const loginLimiter = rateLimit({
  windowMs: 10 * 60 * 1000, // 10 minutes
  max: 5, // 5 attempts per window per IP
  message: { success: false, error: 'Too many login attempts, try again later.' }
});

app.post('/api/auth/login', loginLimiter, async (req, res) => { /* as in Activity 1 */ });
```

Output / Expected behaviour:

A request from an origin other than `http://localhost:5173` is blocked by CORS.
The 6th login attempt within 10 minutes from the same IP receives 429 with the rate-limit message.

Activity 3: Demonstrate and fix NoSQL injection and XSS

Show a vulnerable query and a vulnerable HTML response, then fix both.

Solution:

```
// VULNERABLE: query built directly from user input
app.post('/api/auth/login-unsafe', async (req, res) => {
  // if req.body.password is the object { "$ne": null }, this matches ANY user
  const user = await User.findOne({ username: req.body.username, password: req.body.password });
  res.json({ success: !!user });
});

// FIXED: validate types before querying, and never compare passwords with $-operators
app.post('/api/auth/login-safe', async (req, res) => {
  const { username, password } = req.body;
  if (typeof username !== 'string' || typeof password !== 'string') {
    return res.status(400).json({ success: false, error: 'Invalid input' });
  }
  const user = await User.findOne({ username });
  const valid = user && await bcrypt.compare(password, user.password);
  res.json({ success: !!valid });
});

// VULNERABLE: reflecting untrusted input straight into HTML
app.get('/search-unsafe', (req, res) => {
  res.send(`<h2>Results for: ${req.query.term}</h2>`); // e.g. ?term=<script>...</script>
});

// FIXED: escape before inserting into HTML
function escapeHtml(str) {
  return str.replace(/[&<>"]'/g, ch => ({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[ch]));
}
app.get('/search-safe', (req, res) => {
  res.send(`<h2>Results for: ${escapeHtml(req.query.term || '')}</h2>`);
});

// React note: never do this with untrusted data -
// <div dangerouslySetInnerHTML={{ __html: userInput }} />
// React already escapes {expression} text content by default; prefer that.
```

Output / Expected behaviour:

POST /api/auth/login-unsafe with { "username": "admin", "password": { "\$ne": null } } logs in without a real password -demonstrating the vulnerability.
The same payload against /api/auth/login-safe is rejected with 400 because password is not a string.
GET /search-unsafe?term=<script>alert(1)</script> renders the tag in the page; /search-safe?term=... shows it as harmless escaped text.

3) Graded Lab Tasks

Note: The instructor may adjust these tasks to the level of difficulty and complexity of the solved activities. Tasks should be evaluated in the same lab session.

Lab Task 1: Token expiry and re-login

Reduce the JWT expiresIn from Lab 12 to '2m'. Demonstrate that a request made after expiry is rejected by requireAuth with 401, and that logging in again issues a fresh, working token.

Lab Task 2: Security header/report checklist

Enable helmet on the Lab 12 API and use browser developer tools (Network tab) to list which response headers helmet added (e.g. X-Content-Type-Options, X-Frame-Options). For each header, write one sentence on what it protects against.

Lab Task 3: OWASP Top 10 mapping

Produce a short table mapping each vulnerability class covered in this lab (weak password storage, NoSQL injection, XSS, CORS misconfiguration, missing rate limiting) to its corresponding OWASP Top 10 category, and note one additional Top 10 category not covered in this lab that the students should research on their own.